CarbonPulse AI.

1. The Telemetry Processing Service

This file calculates the raw carbon emissions and handles the gamification points. It uses the exact formulas from DEFRA and Scarborough et al., clamps the saved carbon to zero to prevent negative DB entries, and caps the streak multiplier at 30 days.

TypeScript

```
// services/activityProcessing.ts

// 1. Hardcoded Emission Constants (DEFRA 2024/2025 & Scarborough et al.)[cite: 3]
const                                                string  number
  car  0.21      // Petrol/Diesel average[cite: 3]
  bike  0.00     // Zero-emission[cite: 3]
  walk  0.00     // Zero-emission[cite: 3]
  bus  0.13      // Transit bus[cite: 3]
  train  0.02    // Subway/Light rail[cite: 3]
  ev  0.06       // EV Grid average[cite: 3]


const                                            string  number
  high_meat  7.19         3
  mixed  5.63      // Mapped to medium_meat
  vegetarian  3.81         3
  vegan  2.89            3


const                                      0.82  // India coal-heavy worst-case fallback[cite: 3, 5]

export function processActivity          any                        number           number
  // Step 1: Transport Math
  const                                      'car'
  const                                              0
  const                                                                          0.21  // Conservative
fallback[cite: 5]
  const

  // Step 2: Diet Math[cite: 5]
  const                                 'mixed'
  const                                                                   5.63

  // Step 3: Home Energy Math (assuming fallback if no live API data)[cite: 5]
  const                                      0
  const                                                                          5

  const

  // Step 4: Delta vs Baseline[cite: 5]
  const                                                                  5
  const                          Math        0                    // Clamped to > 0[cite: 5]
```

```
  // Step 5: Gamification Points[cite: 5]
  let            10  // Base log bonus[cite: 5]
  if   'vegetarian'  'vegan'                                  15        5
  if   'walk'  'cycle'  'bike'                              20        5

  if                          0
    const                       Math                 30    0.1  // Capped at 30 days[cite: 5]
                Math                                 5     1                                  5


  return
    totalEmissions  Number                                4    // Prevent floating-point drift[cite: 5]
    carbonSavedKg  Number                                   4            5
    pointsEarned
```

2. The Behavioral Nudge Engine

This file evaluates the user's last 3 logs using a priority cascade to prevent nudge spam[cite: 5]. It applies the psychological frameworks defined in the blueprint.

TypeScript

```
// services/insightEngine.ts

export function generateDailyNudge                  any
  // Priority Cascade: First match wins[cite: 5]

  // Priority 1: Diet Check (Commitment Consistency)[cite: 5]
  const                                                          'high_meat'
  if                          2          5
    return
      id  'nudge_meatless_shift'
      message  "Swap tomorrow's meals to a plant-based diet to completely offset the carbon from
your last commute."         3
      framework  'Gamification + Commitment Consistency'         5
      potentialSavingsKg  4.30 // 4.30 kg/day[cite: 5]



  // Priority 2: Short Car Trips Check (Salience)[cite: 5]
  const                                                                  'car'
                                  15         5
  if                             2          5
    const                                    0
    return
      id  'nudge_subway_substitution'
      message  `A subway trip today avoids enough carbon to charge your smartphone every day
for a year.`         3
      framework  'Salience / Choice Architecture'         5
      potentialSavingsKg  Number                    0.19            2    // Dynamic per trip[cite: 5]
```

```
  // Priority 3: No Green Transport (Default Bias)[cite: 5]
  const                        'walk'  'cycle'  'bus'  'train'  'bike'
  const
  if                                    5
    return
      id  'nudge_bus_transit'
      message  "Taking the bus for your usual route cuts your transit footprint by nearly 40%."
3
      framework  'Default Bias'          5
      potentialSavingsKg  0.80 // 0.80 kg / 10km[cite: 5]



  // Default Fallback Nudge
  return
    id  'nudge_keep_logging'
    message  "Consistency is key. Keep logging your daily actions to unlock advanced insights."
    framework  'Micro-Habit Stacking'          3
    potentialSavingsKg  0
```

3. The LLM System Prompt

This file constructs the highly restrictive prompt intended for your OpenAI or Claude API calls to prevent hallucination[cite: 5].

TypeScript

```
// services/aiCoach.ts

export function buildWeeklyCoachPrompt                any                                    string
5
  const                JSON

  return `
    You are the "Cyberpunk Sustainability Coach" for CarbonPulse AI.
    You are analyzing the following user JSON telemetry data: ${safeData}

    CRITICAL ANTI-HALLUCINATION RULES:
    1. You may ONLY reference numbers, distances, or diet types that explicitly exist within the
provided JSON.[cite: 5]
    2. If a field is null or missing, you must not mention it.[cite: 5]
    3. You must format your response as EXACTLY three punchy, motivational sentences.[cite: 5]

    SENTENCE STRUCTURE MANDATES:
    - Sentence 1 MUST cite a specific numerical figure from the user's data (e.g., "You saved X kg
of CO2e..." or "You traveled Y km...").[cite: 5]
    - Sentence 2 must praise a specific positive action they took.
    - Sentence 3 must provide an actionable directive focusing specifically on improving their
worst emission category, which this week is: [${worstEmissionCategory.toUpperCase()}].[cite: 5]
```

Do not include greetings, sign-offs, or conversational filler. Output only the three sentences.
`

By placing these three logic files into a services/ directory, you have completely fulfilled the backend calculation and AI orchestration logic without adding a single byte to your node_modules.

Activityprocessing

```
/**
 * CarbonPulse AI — Telemetry Processing Service
 * Processes raw daily activity input, calculates kg CO2e, compares against
 * the user's baseline, and returns gamification points for the session.
 *
 * Sourced from DEFRA 2024/2025, EPA, and Scarborough et al. (2014).
 * Zero external dependencies — native TypeScript/JS only.
 */

// ─── Emission Factor Constants ──────────────────────────────────────────────

/** kg CO2e per km (includes Tank-to-Wheel + Well-to-Tank) */
const TRANSPORT_FACTORS_KG_KM: Record<string, number> = {
  walk:  0.00,
  cycle: 0.00,
  car:   0.21, // Average petrol/diesel — DEFRA 2024/2025
  bus:   0.13, // Post-COVID occupancy-adjusted — DEFRA 2024/2025
  train: 0.02, // Electrified metro/light rail — DEFRA 2024/2025
  bike:  0.00, // Motorbike — treated as 0 here; override if needed
} as const;

/** kg CO2e per day — Scarborough et al. (2014), normalised to 2,000 kcal */
const DIET_FACTORS_KG_DAY: Record<string, number> = {
  high_meat:  7.19,
  mixed:      5.63, // maps to "medium meat" in the blueprint
  vegetarian: 3.81,
  vegan:      2.89,
} as const;

// ─── Gamification Constants ─────────────────────────────────────────────────

const POINTS = {
  BASE_LOG:           10,  // awarded for any completed daily log
  VEGAN_BONUS:        25,  // vegan meal choice
```

```
  VEGETARIAN_BONUS:    15,
  ZERO_EMISSION_TRIP:  20,  // walk / cycle / train
  STREAK_MULTIPLIER:    5,  // extra points per streak day (capped at 30 days)
  BELOW_BASELINE_BONUS: 30, // day's total is under the user's personal baseline
} as const;

// ——— Input / Output Types
——————————————————————————————————————————————

export interface ActivityInput {
  transport: {
    mode: "walk" | "cycle" | "bus" | "train" | "car" | "bike";
    distanceKm: number;
  };
  diet: "vegan" | "vegetarian" | "mixed" | "high_meat";
  homeEnergyKwh?: number;       // optional — grid electricity consumed
  gridIntensityKgPerKwh?: number; // optional — from Emissions.dev API; falls back to India
avg
  currentStreak: number;         // consecutive days with a logged activity
  baselineFootprint: number;     // user's personal daily baseline in kg CO2e
}

export interface ProcessedActivity {
  transportEmissionsKg: number;
  dietEmissionsKg: number;
  homeEnergyEmissionsKg: number;
  totalEmissionsKg: number;
  carbonDeltaKg: number;        // negative = saved vs baseline; positive = exceeded
  carbonSavedKg: number;        // clamped to 0 when delta is positive
  pointsEarned: number;
  breakdown: {
    label: string;
    valueKg: number;
    percentage: number;
  }[];
}

// ——— Grid Energy Default
———————————————————————————————————————————————

/**
 * Falls back to the India national average (0.82 kg CO2e/kWh) when no
 * real-time figure is provided, per the blueprint's specification for
 * coal-heavy grids (Consumer Ecology, CEA 2024/2025).
 */
const DEFAULT_GRID_INTENSITY_KG_KWH = 0.82;
```

```
// ——— Core Processing Function ————————————————————————————————

/**
 * Processes a single day's raw activity telemetry.
 *
 * @param input - Raw user-submitted daily activity data
 * @returns A fully typed `ProcessedActivity` result ready for DB persistence
 */
export function processActivity(input: ActivityInput): ProcessedActivity {
  const {
    transport,
    diet,
    homeEnergyKwh = 0,
    gridIntensityKgPerKwh = DEFAULT_GRID_INTENSITY_KG_KWH,
    currentStreak,
    baselineFootprint,
  } = input;

  // ── 1. Transport emissions ——————————————————————————————————————
  const modeKey = transport.mode.toLowerCase();
  const efMode = TRANSPORT_FACTORS_KG_KM[modeKey] ?? 0.21; // default to car if
unknown
  const transportEmissionsKg = roundTo4(transport.distanceKm * efMode);

  // ── 2. Diet emissions ———————————————————————————————————————————
  const dietEmissionsKg = roundTo4(DIET_FACTORS_KG_DAY[diet] ??
DIET_FACTORS_KG_DAY.mixed);

  // ── 3. Home energy emissions (E_elec = C_kWh × EF_grid) ——————————
  const homeEnergyEmissionsKg = roundTo4(homeEnergyKwh * gridIntensityKgPerKwh);

  // ── 4. Totals and delta ——————————————————————————————————————————
  const totalEmissionsKg = roundTo4(
    transportEmissionsKg + dietEmissionsKg + homeEnergyEmissionsKg
  );
  const carbonDeltaKg = roundTo4(totalEmissionsKg - baselineFootprint);
  const carbonSavedKg = carbonDeltaKg < 0 ? roundTo4(Math.abs(carbonDeltaKg)) : 0;

  // ── 5. Breakdown for SVG donut chart ————————————————————————————
  const breakdown = buildBreakdown(
    transportEmissionsKg,
    dietEmissionsKg,
```

```
    homeEnergyEmissionsKg,
    totalEmissionsKg
  );

  // ── 6. Gamification points ──────────────────────────────────────────
  const pointsEarned = calculatePoints(
    transport.mode,
    diet,
    carbonDeltaKg,
    currentStreak
  );

  return {
    transportEmissionsKg,
    dietEmissionsKg,
    homeEnergyEmissionsKg,
    totalEmissionsKg,
    carbonDeltaKg,
    carbonSavedKg,
    pointsEarned,
    breakdown,
  };
}

// ─── Gamification Points Calculator ──────────────────────────────────

function calculatePoints(
  mode: ActivityInput["transport"]["mode"],
  diet: ActivityInput["diet"],
  carbonDeltaKg: number,
  streak: number
): number {
  let points = POINTS.BASE_LOG;

  // Diet bonus
  if (diet === "vegan") points += POINTS.VEGAN_BONUS;
  else if (diet === "vegetarian") points += POINTS.VEGETARIAN_BONUS;

  // Zero-emission transport bonus
  if (mode === "walk" || mode === "cycle" || mode === "train") {
    points += POINTS.ZERO_EMISSION_TRIP;
  }

  // Below-baseline bonus
  if (carbonDeltaKg < 0) points += POINTS.BELOW_BASELINE_BONUS;
```

```
  // Streak multiplier (capped at 30 days to prevent runaway accumulation)
  const cappedStreak = Math.min(streak, 30);
  points += cappedStreak * POINTS.STREAK_MULTIPLIER;

  return points;
}

// ——— Breakdown Builder
——————————————————————————————————————————————

function buildBreakdown(
  transport: number,
  diet: number,
  energy: number,
  total: number
): ProcessedActivity["breakdown"] {
  if (total === 0) {
    return [
      { label: "Transport",   valueKg: 0, percentage: 0 },
      { label: "Diet",        valueKg: 0, percentage: 0 },
      { label: "Home Energy", valueKg: 0, percentage: 0 },
    ];
  }

  const pct = (v: number) => roundTo4((v / total) * 100);

  return [
    { label: "Transport",   valueKg: transport, percentage: pct(transport) },
    { label: "Diet",        valueKg: diet,      percentage: pct(diet) },
    { label: "Home Energy", valueKg: energy,    percentage: pct(energy) },
  ];
}

// ——— Utility
——————————————————————————————————————————————
————————————

/** Rounds a float to 4 decimal places to avoid floating-point drift in DB. */
function roundTo4(value: number): number {
  return Math.round(value * 10_000) / 10_000;
}
```

Insightengine

```
/**
 * CarbonPulse AI — Behavioral Nudge Engine
 * Evaluates the last 3 days of user activity telemetry and emits a single,
 * contextually relevant behavioral nudge using Fogg's B=MAP model.
```

```
 *
 * Psychological frameworks: Nudge Theory (Thaler & Sunstein),
 * Fogg Behavior Model, and the micro-action matrix from the blueprint.
 * Zero external dependencies.
 */

// ——— Types ——————————————————————————————————————————————————————————————

/** Shape of a single stored activity log — mirrors the ActivityLog Mongoose schema. */
export interface RecentLog {
  date: string | Date;
  transport: {
    mode: "walk" | "cycle" | "bus" | "train" | "car" | "bike";
    distanceKm: number;
  };
  diet: "vegan" | "vegetarian" | "mixed" | "high_meat";
  totalCarbonSaved: number;
}

export type PsychologicalFramework =
  | "GAMIFICATION"
  | "SALIENCE"
  | "LOSS_AVERSION"
  | "COMMITMENT_CONSISTENCY"
  | "SOCIAL_PROOF"
  | "DEFAULT_BIAS";

export interface BehavioralNudge {
  id: string;
  headline: string;
  message: string;
  framework: PsychologicalFramework;
  potentialSavingsKg: number;
  /** Data-driven context used to compose the nudge (useful for logging/A-B testing) */
  triggerContext: Record<string, unknown>;
}

// ——— Emission Factor Constants (mirrored from activityProcessing.ts) ——————————

/** kg CO2e per day — Scarborough et al. (2014) */
const DIET_KG_DAY = {
  high_meat:  7.19,
  mixed:      5.63,
  vegetarian: 3.81,
  vegan:      2.89,
} as const;
```

```
/** kg CO2e per km */
const TRANSPORT_KG_KM = {
  car:   0.21,
  bus:   0.13,
  train: 0.02,
  walk:  0.00,
  cycle: 0.00,
  bike:  0.00,
} as const;

// ——— Core Nudge Engine ———————————————————————————————————————————

/**
 * Evaluates up to the last 3 activity logs and returns the single highest-
 * priority nudge. Returns `null` if no trigger condition is met.
 *
 * Priority order (first match wins):
 *   1. Consecutive high-meat logging → "Meatless Monday" challenge
 *   2. Repeated short car trips → subway substitution
 *   3. Any car usage with available greener mode → transit default-bias nudge
 *
 * @param recentLogs - Array of the user's most recent activity logs (newest first)
 */
export function generateDailyNudge(
  recentLogs: RecentLog[]
): BehavioralNudge | null {
  // Normalise to the 3 most recent entries, newest first
  const logs = recentLogs.slice(0, 3);

  if (logs.length === 0) return null;

  // ── Trigger 1: Consecutive High-Meat Days ——————————————————————
  const meatlessNudge = evaluateMeatlessMonday(logs);
  if (meatlessNudge) return meatlessNudge;

  // ── Trigger 2: Repeated Short Car Trips (< 15 km) ——————————————
  const subwayNudge = evaluateSubwaySubstitution(logs);
  if (subwayNudge) return subwayNudge;

  // ── Trigger 3: Any Car Usage → Bus Transit Default ——————————————
  const transitNudge = evaluateBusTransitDefault(logs);
  if (transitNudge) return transitNudge;
```

```
  return null;
}

// ——— Trigger Evaluators ——————————————————————————————————————————

/**
 * TRIGGER 1 — "Meatless Monday" Challenge
 * Framework: Gamification + Commitment Consistency
 *
 * Fires when the user has logged `high_meat` for 2 or more of the last 3 days.
 * Savings = high_meat daily factor − vegan daily factor (4.30 kg CO2e/day).
 */
function evaluateMeatlessMonday(logs: RecentLog[]): BehavioralNudge | null {
  const highMeatDays = logs.filter((l) => l.diet === "high_meat").length;

  if (highMeatDays < 2) return null;

  const savingsKg = roundTo2(DIET_KG_DAY.high_meat - DIET_KG_DAY.vegan); // 4.30

  return {
    id: "nudge_meatless_monday",
    headline: "🌱 Meatless Monday — Unlock a Carbon Badge",
    message:
      `You've logged a high-meat diet ${highMeatDays} days in a row. ` +
      `Swapping tomorrow's meals to plant-based could save you ${savingsKg} kg CO₂e — ` +
      `enough to completely offset your last car commute. ` +
      `Complete the challenge and earn +50 bonus points.`,
    framework: "GAMIFICATION",
    potentialSavingsKg: savingsKg,
    triggerContext: { highMeatDays, logsEvaluated: logs.length },
  };
}

/**
 * TRIGGER 2 — Subway Substitution (Short Car Trips)
 * Framework: Salience / Choice Architecture
 *
 * Fires when at least 2 of the last 3 logs show a car trip under 15 km.
 * Savings per trip = distance × (car EF − train EF).
 */
function evaluateSubwaySubstitution(logs: RecentLog[]): BehavioralNudge | null {
  const shortCarTrips = logs.filter(
    (l) => l.transport.mode === "car" && l.transport.distanceKm < 15
  );

  if (shortCarTrips.length < 2) return null;
```

```
  // Use the most recent short trip's distance for the savings calculation
  const latestTrip = shortCarTrips[0];
  const savingsKg = roundTo2(
    latestTrip.transport.distanceKm * (TRANSPORT_KG_KM.car -
TRANSPORT_KG_KM.train)
  );
  const avgDistanceKm = roundTo2(
    shortCarTrips.reduce((sum, l) => sum + l.transport.distanceKm, 0) /
      shortCarTrips.length
  );

  return {
    id: "nudge_subway_substitution",
    headline: "🚇 Switch Lanes — Your Commute's Carbon Is Stacking Up",
    message:
      `You've driven ${shortCarTrips.length} short trips (avg ${avgDistanceKm} km) this week.
` +
      `Taking the subway instead of your car on a ${latestTrip.transport.distanceKm} km route `
+
      `avoids ${savingsKg} kg CO₂e — that's enough energy to charge a smartphone every
day for a year.`,
    framework: "SALIENCE",
    potentialSavingsKg: savingsKg,
    triggerContext: {
      shortCarTripCount: shortCarTrips.length,
      avgDistanceKm,
      latestDistanceKm: latestTrip.transport.distanceKm,
    },
  };
}

/**
 * TRIGGER 3 — Bus Transit Default Bias
 * Framework: Default Bias
 *
 * Fires when any log in the window shows car usage, and no greener mode has
 * been used in the same window. Savings are calculated over 10 km (blueprint).
 * Savings = 10 km × (car EF − bus EF) = 0.80 kg CO₂e.
 */
function evaluateBusTransitDefault(logs: RecentLog[]): BehavioralNudge | null {
  const hasCarUsage = logs.some((l) => l.transport.mode === "car");
  const hasGreenMode = logs.some(
    (l) =>
      l.transport.mode === "train" ||
      l.transport.mode === "bus" ||
      l.transport.mode === "walk" ||
      l.transport.mode === "cycle"
  );
```

```
  // Only nudge if the user is exclusively using the car — don't nag switchers
  if (!hasCarUsage || hasGreenMode) return null;

  const REFERENCE_DISTANCE_KM = 10;
  const savingsKg = roundTo2(
    REFERENCE_DISTANCE_KM * (TRANSPORT_KG_KM.car -
TRANSPORT_KG_KM.bus)
  ); // 0.80

  return {
    id: "nudge_bus_transit_default",
    headline: "🚌 Make Transit Your Default — Cut 40% Off Your Commute Footprint",
    message:
      `All of your recent trips have been by car. ` +
      `Choosing the bus for a ${REFERENCE_DISTANCE_KM} km route cuts your transit ` +
      `carbon footprint by nearly 40%, saving ${savingsKg} kg CO₂e per trip. ` +
      `Set transit as your default and watch your streak climb.`,
    framework: "DEFAULT_BIAS",
    potentialSavingsKg: savingsKg,
    triggerContext: {
      exclusiveCarDays: logs.filter((l) => l.transport.mode === "car").length,
      referenceDistanceKm: REFERENCE_DISTANCE_KM,
    },
  };
}

// —— Utility
————————————————————————————————————————————————————————————————————————————————

function roundTo2(value: number): number {
  return Math.round(value * 100) / 100;
}
```

Aicoach · TS

```
/**
 * CarbonPulse AI — Dynamic AI Coach System Prompt
 *
 * Usage: pass `CARBON_COACH_SYSTEM_PROMPT` as the `system` parameter
 * in your LLM API call, then inject the user's weekly JSON as the
 * `user` message using `buildWeeklyCoachPrompt()`.
 *
 * The prompt enforces:
 *   - Cyberpunk Sustainability Coach persona
 *   - Exactly 3 output sentences
```

```
 *   - Strict grounding in the provided JSON (no hallucination)
 *   - No markdown, no lists, no caveats
 */

// ——— System Prompt
——————————————————————————————————————————————————————————————————————
————

export const CARBON_COACH_SYSTEM_PROMPT = `
You are PULSE — a Cyberpunk Sustainability Coach embedded inside the CarbonPulse AI
platform.
Your communication style is direct, punchy, and motivational. You speak like a veteran
hacker
who cares deeply about the planet: sharp, confident, and data-obsessed.
You never use filler phrases like "Great job!" or "I'm proud of you."
You never use markdown formatting, bullet points, numbered lists, or headers.

YOUR ONLY JOB:
Analyse the weekly carbon footprint JSON object provided in the user message and return
EXACTLY 3 sentences — no more, no fewer. Each sentence must be punchy, motivational,
and no longer than 25 words.

SENTENCE STRUCTURE RULES:
- Sentence 1: State one concrete, specific insight derived directly from the provided JSON
numbers.
  You MUST reference an actual figure from the data (e.g., a kg CO2e value, a streak count,
  or a percentage). Do NOT invent or estimate any figure.
- Sentence 2: Frame one behaviour from the data as either a win to amplify or a loss to
recover.
  Use the Loss Aversion or Gamification framing. Reference specific data.
- Sentence 3: Deliver one precise, actionable directive the user can execute within 24 hours.
  It must be grounded in the data's weakest category (the emission source with the highest
kg CO2e).

HALLUCINATION PREVENTION — CRITICAL RULES:
- You may ONLY reference numbers, modes, diet types, and categories that appear explicitly
  in the JSON object. Do NOT infer, estimate, or fabricate any value.
- If a field is missing or null in the JSON, do NOT reference that field.
- If the data is insufficient to form a grounded sentence, write: "Insufficient data to generate
  a coaching insight for this period." and stop.
- You must not claim the user did something the JSON does not confirm.
- Do not use comparative language ("better than average", "most users") unless the JSON
  contains a benchmark value to support it.

OUTPUT FORMAT:
Return only the 3 raw sentences separated by a single space. No labels. No punctuation
beyond
sentence-ending periods or exclamation marks. No preamble. No sign-off.
```

```
`.trim();

// ——— Weekly Data Types ——————————————————————————————————————————

export interface DailyLogSummary {
  date: string;                 // ISO 8601
  transportEmissionsKg: number;
  dietEmissionsKg: number;
  homeEnergyEmissionsKg: number;
  totalEmissionsKg: number;
  carbonSavedKg: number;
  transport: {
    mode: string;
    distanceKm: number;
  };
  diet: string;
  pointsEarned: number;
}

export interface WeeklyCoachData {
  userId: string;
  weekStartDate: string;        // ISO 8601
  weekEndDate: string;          // ISO 8601
  dailyLogs: DailyLogSummary[];
  weeklyTotals: {
    transportEmissionsKg: number;
    dietEmissionsKg: number;
    homeEnergyEmissionsKg: number;
    totalEmissionsKg: number;
    totalCarbonSavedKg: number;
    totalPointsEarned: number;
  };
  streak: number;
  baselineFootprintKgPerDay: number;
  weeklyBaselineKg: number;     // baselineFootprintKgPerDay × 7
}

// ——— Prompt Builder ——————————————————————————————————————————————
——

/**
 * Builds the user-role message to accompany `CARBON_COACH_SYSTEM_PROMPT`.
 * Serialises the weekly data as compact JSON to minimise token usage.
 *
 * @param data - Aggregated weekly carbon data for a single user
 * @returns The fully formatted user message string
```

```
 */
export function buildWeeklyCoachPrompt(data: WeeklyCoachData): string {
  // Validate that we have enough logs to avoid trivial outputs
  if (!data.dailyLogs || data.dailyLogs.length === 0) {
    return JSON.stringify({
      error: "NO_LOGS",
      message: "No activity logs recorded for this week.",
    });
  }

  // Derive the highest-emission category so the prompt has a concrete anchor
  const totals = data.weeklyTotals;
  const categories: { label: string; kg: number }[] = [
    { label: "transport",   kg: totals.transportEmissionsKg },
    { label: "diet",        kg: totals.dietEmissionsKg },
    { label: "home_energy", kg: totals.homeEnergyEmissionsKg },
  ];
  const worstCategory = categories.reduce((a, b) => (a.kg > b.kg ? a : b));

  // Compute vs-baseline delta for the week
  const baselineDeltaKg = roundTo2(
    totals.totalEmissionsKg - data.weeklyBaselineKg
  );

  const payload = {
    period: {
      start: data.weekStartDate,
      end:   data.weekEndDate,
      daysLogged: data.dailyLogs.length,
    },
    streak: data.streak,
    weeklyTotals: totals,
    baselineDeltaKg,
    worstEmissionCategory: worstCategory,
    dailySummary: data.dailyLogs.map((log) => ({
      date:          log.date,
      totalKg:       log.totalEmissionsKg,
      savedKg:       log.carbonSavedKg,
      transportMode: log.transport.mode,
      distanceKm:    log.transport.distanceKm,
      diet:          log.diet,
      points:        log.pointsEarned,
    })),
  };

  return (
    "Analyse this weekly carbon footprint JSON and return exactly 3 coaching sentences:\n\n"
+
```

```
    JSON.stringify(payload, null, 0)
  );
}

// ——— Utility
————————————————————————————————————————————————————————————————————————————————

function roundTo2(value: number): number {
  return Math.round(value * 100) / 100;
}

// ——— Usage Example (for reference — delete before production)
————————————————————

/*
import Anthropic from "@anthropic-ai/sdk"; // or use native fetch
import { CARBON_COACH_SYSTEM_PROMPT, buildWeeklyCoachPrompt,
WeeklyCoachData } from "./aiCoach";

async function getWeeklySummary(data: WeeklyCoachData): Promise<string> {
  const response = await fetch("https://api.anthropic.com/v1/messages", {
    method: "POST",
    headers: {
      "Content-Type":      "application/json",
      "x-api-key":         process.env.ANTHROPIC_API_KEY!,
      "anthropic-version": "2023-06-01",
    },
    body: JSON.stringify({
      model:      "claude-sonnet-4-20250514",
      max_tokens: 200,
      system:     CARBON_COACH_SYSTEM_PROMPT,
      messages: [
        { role: "user", content: buildWeeklyCoachPrompt(data) }
      ],
    }),
  });

  const json = await response.json();
  return json.content?.[0]?.text ?? "";
}
*/
```